Building femm-qt on FreeBSD

D. Meeker

October 23, 2026

1 Overview

femm-qt is a cross-platform port of FEMM (Finite Element Method Magnetics) using Qt 6 for the GUI and CMake/Ninja for the build system. This document describes how to build it from source on FreeBSD. The instructions target FreeBSD 14, but should work with minor changes on 13.x and later.

The C++ source and the CMake build are platform-generic: there is no Linux-specific code, and the only places that differ from the Linux build are the toolchain (FreeBSD's base compiler is clang, not g++), the package manager (`pkg`), and the install prefix (`/usr/local`, a.k.a. `LOCALBASE`). The dedicated `freebsd_build.sh` script handles those differences for you.

The build produces six executables:

- `femm_gui` — the main GUI application
- `femm_headless` — headless IPC server (no GUI)
- `fkn` — magnetics solver
- `belasolv` — electrostatics solver
- `hsolv` — heat flow solver
- `csolv` — current flow solver

The commands below use `sudo` for administrative actions. GhostBSD ships `sudo` preconfigured; on a base FreeBSD system, either install it (`pkg install sudo`, run as root) and add your user to the `wheel` group, or simply run the privileged commands from a root shell (`su -`). The base system's `doas` also works once you create `/usr/local/etc/doas.conf`.

2 Prerequisites

2.1 Build Tools

FreeBSD ships clang, the C++ standard library, and the linker in the base system, so no separate "build-essential" package is required. Install CMake, Ninja, git, and bash (the build script is a bash script):

```
sudo pkg install cmake ninja git bash
```

This provides `cmake`, `ninja`, `git`, and `bash`; the C++ compiler is already present as `cc`/`c++`.

2.2 Qt 6

Install the Qt 6 libraries and development files:

```
sudo pkg install qt6
```

The `qt6` metapackage pulls in everything, including the `moc`, `rcc`, and `uic` code generators that the build needs. If you prefer a minimal install, `qt6-base` is usually sufficient for this project. Qt 6 installs under `/usr/local`; no separate Qt installer or online account is needed.

To verify the installation:

```
qmake6 --version
```

3 Getting the Source

If you have a source zip, unzip it:

```
unzip femm-qt-src.zip
cd femm-qt
```

Or, if the source is hosted on GitHub, clone the repository:

```
git clone <repository-url> femm-qt
cd femm-qt
```

4 Building

From the `femm-qt` directory, run the build script:

```
./freebsd_build.sh
```

This checks that the build tools are present, configures the project with CMake (pointing it at `/usr/local` so Qt 6 is found), and builds it with Ninja. All executables are placed in `build/bin/`.

To wipe the build directory and rebuild from scratch, pass `clean`:

```
./freebsd_build.sh clean
```

If your Qt 6 packages live somewhere other than `/usr/local`, set `LOCALBASE` before running the script:

```
LOCALBASE=/opt/local ./freebsd_build.sh
```

If the build fails because CMake cannot find Qt 6, see *Manual configuration* under Troubleshooting.

5 Installing

You can run femm-qt from `build/bin` indefinitely; nothing requires an install. To put it somewhere permanent instead:

```
./freebsd_build.sh install
```

This installs what is already built — it does not rebuild — into your home directory:

```
~/.local/opt/femm-qt/   the program, its solvers, the material
                        libraries, the bindings, and a pristine
                        set of the examples
~/.local/bin/           femm_gui, femm_headless, the four solvers
                        (fkn, belasolv, hsolv, csolv), lua4, and
```

```
                          femm-lua
~/.local/share/           menu entry and icon
~/Documents/FEMM/
       examples/          your copy of the examples
       docs/              the manual and the scripting references
```

The solvers are linked as commands of their own because each is usable that way: `fkn model` solves `model.fem` and writes `model.ans`, with `belasolv`, `hsolv` and `csolv` doing the same for electrostatics, heat flow and current flow — which is what you want from a Makefile or a batch queue. `lua4` is a stock Lua 4 interpreter and runs directly too.

The examples are installed twice on purpose. The program directory is out of the way — and on Ubuntu worse than that, since the file manager hides `~/.local` outright — so a second copy goes somewhere you can actually open, under whatever your desktop calls *Documents*. The shipped examples in it are replaced on every install, so that a corrected example reaches you rather than only new users — and because *running* an example rewrites its `.fem` (analysis saves the document first), which makes any "has this been edited?" test useless here. Anything else you keep in that directory is untouched: only the files femm-qt ships are written. Work in a copy, which is what every example script does on its second line.

FEMM then appears in the application menu. For `femm_gui` to work as a command, `~/.local/bin` has to be on your `PATH`, and **the installer does not put it there without being asked**. Your shell startup files are your configuration — kept in a dotfiles repository, synced between machines, or managed for you on an institutional login — and an installer editing one uninvited produces a confusing diff at best and a change that is silently reverted at worst.

So it checks, and tells you. If the directory is already on `PATH` it says nothing at all. On Debian and Ubuntu the stock `~/.profile` already adds `~/.local/bin` when it exists, so it will be there at your next login and the installer leaves well alone. Otherwise it prints the one line to add and which file to add it to.

If you would rather it did the edit, ask for it:

```
./freebsd_build.sh install --modify-path
```

which appends a single marked block, once — running it again finds the block and adds nothing.

Which file that is depends on your shell, and so does when the change takes effect. `~/.profile` is read at login by the desktop session, and every terminal inherits the environment from there, so a bash or `sh` user must log out and back in — opening a new terminal window is *not* enough. `~/.zshrc` and fish's `config.fish` are read by every interactive shell, so there a new terminal really is sufficient. Under `csh` or `tcsh` nothing is written at all: those read none of these files and use a different syntax entirely, so the installer prints the `set path` line for `~/.cshrc` rather than leave a line that would never run.

Whatever you do, it cannot fix the shell you ran it from — no program can change the environment of the shell that launched it.

Pass a different prefix if you would rather it went elsewhere:

```
./freebsd_build.sh install /opt/femm
```

It installs into your home rather than `/usr/local` on purpose, and it *refuses to run as root*. This script sets up one user's environment: besides the program directory it puts examples and documents in your home folder and can edit your shell profile. Run under `doas` or `sudo` those would land in `/root` while the program itself was shared, so every other user on the machine would get a program with no examples and no documents — and it would look as though it had worked.

For a system-wide install, or to build a port or package, use the CMake install rules instead:

```
cmake --install build --prefix /usr/local
```

That produces an ordinary `bin/` and `share/femm/` layout and honours `DESTDIR`. It is safe where this script is not because nothing FEMM writes lives in the installation any more: your material libraries and your own Python modules are kept per-user, under `~/.local/share/FEMM/`, and are found there at run time.

Uninstalling

```
./freebsd_build.sh uninstall
```

It removes the program directory, the commands, the menu entry, the icon and the documents, and takes out the `PATH` line if it was the one that added it. Pass the same prefix you installed with if it was not the default, and `-y` to skip the confirmation.

It works from a manifest the install leaves in the program directory rather than from a list of paths worked out afresh, because several of those paths are not reliably re-derivable later — which solver commands were linked, whether a profile was edited, and above all where the documents went, since that comes from `xdg-user-dir` and can be reconfigured. A program directory with no manifest is refused rather than guessed at, and a command in `~/.local/bin` that does not point into the program directory is left alone: a name like `lua4` could easily belong to something else.

Your copy of the examples is never removed. Those are yours, an install never overwrites them, and a script that deletes a directory somebody has been working in is not one to run casually.

6 Running

Launch the GUI:

```
./build/bin/femm_gui
```

Or the headless server:

```
./build/bin/femm_headless
```

If you installed as above, use the copies on your `PATH` instead, from any directory:

```
femm_gui
femm_headless
```

6.1 Setting FEMM_DIR

The language bindings (OctaveFEMM, MathFEMM, pyFEMM, JuliaFEMM) locate the FEMM executables via the `FEMM_DIR` environment variable. Set it to the `bin` directory. For `sh`/`bash` (in `~/.profile` or `~/.bashrc`):

```
export FEMM_DIR=/path/to/femm-qt/build/bin
```

For `csh`/`tcsh` (the default root shell; in `~/.cshrc`):

```
setenv FEMM_DIR /path/to/femm-qt/build/bin
```

If you installed, you can skip all of this. Each binding looks for `femm_gui` and `femm_headless` in three places, in order: `FEMM_DIR`, then beside its own directory, then *every directory on `PATH`* — and the install puts them on `PATH`. So the bindings work with nothing set at all. `FEMM_DIR` is how you override that: when running from a build tree that was never installed, or when you have several builds and need to say which. Being first in the list, it beats both the relative search and `PATH`.

The binding files themselves are in `bindings/` under the program directory — `mfiles` for OctaveFEMM, and `mathfemm`, `scifemm`, `juliafemm`, `pyfemm` and `luafemm` beside it.

6.2 Desktop Integration (Optional)

The `install` step above does this for you, with an icon. The recipe below is for running in place out of `build/bin`.

If you run a desktop environment (KDE, XFCE, etc.), add femm-qt to your application menu by creating a `.desktop` file:

```
cat > ~/.local/share/applications/femm-qt.desktop << 'EOF'
[Desktop Entry]
Name=FEMM
Comment=Finite Element Method Magnetics
Exec=/path/to/femm-qt/build/bin/femm_gui
Type=Application
Categories=Science;Engineering;
EOF
```

7 Using with Octave

MATLAB is not available natively on FreeBSD, so Octave is the supported route for the *.m language bindings.

Install Octave and a JDK (OctaveFEMM uses Java for socket communication):

```
sudo pkg install octave openjdk21
```

Find the path to Java:

```
readlink -f $(which java)
```

This usually points under `/usr/local/openjdk21`. Set `JAVA_HOME` and `FEMM_DIR` in your shell startup file — for `sh`/`bash`:

```
export JAVA_HOME=/usr/local/openjdk21
```

(`FEMM_DIR` used to be needed here too. It no longer is: `openfemm` searches `PATH` for `femm_gui` and `femm_headless`, and an install puts them there. Set it only to choose between several builds.) Run Octave and confirm Java is available:

```
version -java
```

Then add the OctaveFEMM mfiles to Octave's path:

```
addpath("~/.local/opt/femm-qt/bindings/mfiles")
savepath
```

or, working from the source tree instead of an install, `addpath("/path/to/femm-qt/octavefemm")`.

This is a *different* search path from the one above, and the two are easy to confuse. Octave's path is how Octave finds `openfemm.m`; the system `PATH` is how `openfemm.m` then finds `femm_gui`. Only the second is automatic. The install prints the `addpath` line for every language when it finishes.

A fuller OctaveFEMM guide is installed at `~/Documents/FEMM/docs/octavefemm.md`.

8 Using with Python

The Python interface is/will be available as pyfemm-qt. Install Python and pip first:

```
sudo pkg install python3 py311-pip
```

Then install pyfemm from PyPI:

```
pip install pyfemm-qt
```

To install the version that shipped with this source instead, change to the `pyfemm` directory — in the source tree, or `~/.local/opt/femm-qt/bindings/pyfemm` after an install — and install in editable mode:

```
cd pyfemm
pip install -e .
```

`-e` records the *absolute path* of that directory, so an editable install breaks if the directory is later moved or deleted. Point it at the installed copy, not at a build tree you might clean. Like recent Linux distributions, FreeBSD marks the system Python as externally managed (PEP 668), so pip may refuse to install into it. The cleanest fix is a virtual environment:

```
python3 -m venv ~/femm-venv
. ~/femm-venv/bin/activate
pip install -e .
```

Since pyfemm is a single file with no dependencies, overriding the check is also safe:

```
pip install -e . --break-system-packages
```

A fuller pyFEMM guide — multi-instance use, headless mode, connecting to an instance that is already running — is installed at `~/Documents/FEMM/docs/pyfemm.md`.

9 Troubleshooting

9.1 Manual configuration

The build script just wraps the standard CMake and Ninja steps. To run them by hand:

```
mkdir build && cd build
cmake .. -G Ninja -DCMAKE_BUILD_TYPE=Release \
    -DCMAKE_PREFIX_PATH=/usr/local
ninja
```

The `CMAKE_PREFIX_PATH=/usr/local` hint lets find_package locate the Qt 6 packages installed by `pkg`.

9.2 "env: bash: No such file or directory"

The build script is a bash script, but bash is not part of the FreeBSD base system. Install it:

```
sudo pkg install bash
```

9.3 Qt not found

If CMake reports that it cannot find Qt6, check that the package is installed:

```
pkg info -x qt6
```

If it is installed but not found, pass `-DCMAKE_PREFIX_PATH` pointing at the Qt 6 CMake config directory, typically `/usr/local/lib/cmake/Qt6`:

```
cmake .. -G Ninja -DCMAKE_BUILD_TYPE=Release \
    -DCMAKE_PREFIX_PATH=/usr/local/lib/cmake/Qt6
```

9.4 Qt found, but CMake stops inside it

A different failure from the one above, and the advice above does not fix it. CMake locates Qt 6 quite happily and then dies part-way through configuring, before compiling anything:

```
CMake Error at /usr/local/lib/cmake/Qt6Gui/
              Qt6QICNSPluginTargetsPrecheck.cmake:2:
  Unknown CMake command "_qt_internal_should_include_targets".
```

Nothing is wrong with the source tree, and `-DCMAKE_PREFIX_PATH` will not help: Qt was found. The Qt 6 packages are at *different versions*. The helper that error names comes from `qt6-base`, while the file calling it is installed by a newer plugin package, so the two disagree about what exists.

This happens on an installation image more than a month or two old. Whatever Qt pieces the image shipped stay at their original version, `pkg install qt6` fetches the current ones on top, and the result is a mixture. Confirm it by looking for more than one version:

```
pkg info -x qt6
```

The fix is to update the system first and build afterwards. On GhostBSD use Update Station, which creates a boot environment as it goes; on FreeBSD, `sudo pkg upgrade`. On a three-month-old image expect several hundred packages.

Two practical notes, because this bites on a virtual machine. `sudo pkg upgrade -n` prints what the upgrade needs without changing anything — `pkg` stages every package under `/var/cache/pkg` before installing, so peak usage is roughly the download plus the installed growth, and a 15 GB disk is not enough for a Qt-sized upgrade. And a ZFS boot environment taken by hand beforehand makes the space problem *worse* rather than better: the snapshot pins the old blocks, so nothing the upgrade rewrites can be freed while it exists. With room to spare it is worth having; on a tight disk it is what runs you out.

9.5 Missing OpenGL headers

On minimal installations or VMs without a desktop, Qt may need the Mesa OpenGL libraries and DRI drivers:

```
sudo pkg install mesa-libs mesa-dri libglvnd
```

9.6 VirtualBox Guest Additions

If running in a VirtualBox VM, install the FreeBSD guest additions for proper display scaling and shared folders:

```
sudo pkg install virtualbox-ose-additions
```

Enable the guest services so they start at boot:

```
sudo sysrc vboxguest_enable="YES"
sudo sysrc vboxservice_enable="YES"
```

Reboot after installation. To use a shared folder, add your user to the `wheel`/`operator` group as appropriate and mount it with the `vboxsf` filesystem, e.g.:

```
sudo mount -t vboxvfs <share-name> /mnt
```

9.7 MESA / OpenGL errors in VirtualBox

VirtualBox's virtual GPU may not support the OpenGL version Qt expects, producing errors like "ZINK: failed to choose pdev" or "failed to create dri2 screen." Two fixes:

1. Enable 3D Acceleration in VirtualBox (Settings → Display → check "Enable 3D Acceleration" and raise the video memory to at least 128 MB in Expert mode), or
2. Force Qt to use software rendering by adding (for `sh`/`bash`)

   ```
   export LIBGL_ALWAYS_SOFTWARE=1
   ```

   to your shell startup file (or `setenv LIBGL_ALWAYS_SOFTWARE 1` for csh/tcsh).

9.8 SciFEMM does not work

SciFEMM is the one binding with no working configuration on FreeBSD, and the cause is in Scilab rather than in femm-qt.

Scilab has no native sockets, so SciFEMM talks to femm-qt through `java.net.Socket`. That needs a Scilab with a JVM, and only one of the two command-line binaries has one:

- `scilab-cli` is `-nwni` mode — no window, no Java. It runs, then refuses the first thing SciFEMM does: *"Scilab 'jimport' function disabled in -nogui or -nwni mode"*.
- `scilab-adv-cli` has the JVM, and dies on startup.

The failure is not specific to SciFEMM. It reproduces with no femm-qt code involved at all:

```
scilab-adv-cli -quit -e "disp(1+1)"
```

How it dies depends on the Scilab version. With `scilab-2026.1.0_1` (Scilab branch 2026.1), tested on GhostBSD, FreeBSD 15.0-RELEASE-p13, it aborts while setting up its Java library path, before Scilab itself starts:

```
Exception in thread "main" java.lang.UnsupportedOperationException: set
    at org.scilab.modules.jvm.LibraryPath.addPath(Unknown Source)
Signal: Abort trap (6)
Could not access to the Main Scilab Class:
```

with exit status 51, the same on every run. The earlier `scilab-6.1.1_30` instead printed a fatal error and `Signal: Segmentation fault (11)`, `Address not mapped`, failing at address `0x0`.

Two things this is *not*, both checked:

- **Not the JVM on the machine.** OctaveFEMM also uses Java for its socket communication — see *Using with Octave* above, which installs a JDK for exactly that reason — and works.
- **Not the OpenGL problem** described in the previous subsection. The `libEGL/DRI3` warnings do appear before the failure, but setting `LIBGL_ALWAYS_SOFTWARE=1` removes those warnings and the failure is unchanged. They are noise.

Nor is it simply an old port. This section once concluded that `scilab-6.1.1_30`, several years behind the releases that work on Windows and Linux, was the cause. The port has since reached the 2026 release, and Scilab still does not start. Scilab's own project publishes no FreeBSD binaries.

Stock FreeBSD's own `scilab` package has not been tested; if it works there, the difference is the build environment rather than the source. The other five bindings — OctaveFEMM, pyFEMM, JuliaFEMM, LuaFEMM and MathFEMM — are unaffected.